ECE 385

SPRING 2025

Experiment 7

SOC with NIOS II in SystemVerilog

Artha Amanda Sidauruk
(3220300399)
Cristina Natalia Lumban Tobing
(3220300398)
Wednesday, April 9, 2025.
TEAM 44. D-231
TA: Qiyang Wu

1. Introduction

The NIOS-II processor is a 32-bit CPU designed for Intel/Altera FPGAs, such as Cyclone IV, serving as a foundation for System-on-Chip (SOC) designs. In this lab, NIOS-II acts as a programmable controller, interfacing with memory-mapped peripherals such as SDRAM for storing programs and PIO blocks for managing switch inputs and LED outputs. By executing C programs, it efficiently handles tasks like reading switch values, summing them up, and displaying the result on LEDs. So, by utilizing both Nios-II and Cyclone IV FPGA, the overall system can accomplish the 8-bit inputs accumulator program and also LED blinking. This setup takes advantage of the FPGA's parallel hardware processing abilities to optimize I/O operations. By integrating the NIOS-II with custom logic, this flexible SoC design exemplifies a powerful combination of software management and hardware acceleration tailored for embedded systems.

2. Written Description and Diagrams of NIOS-II System

2.1. Summary of Operation

2.1.1. Hardware Components

The following are the hardware components used in this lab. Their connection can be seen in the figure 1:

- **Nios II Processor, specifically Nios II/e.**
  The Nios II Processor is a soft-core CPU that functions as a system controller. It has a 32-bit modified Harvard RISC architecture. The Nios II processor is able to execute high-level programming language, such as C. The Nios II processor uses a 32-bit Avalon memory-mapped bus to interface to the memory and PIO, peripherals to handle user input/output tasks.
- **On-Chip Memory**
  Chip memory is a small on-chip16-byte RAM placeholder block. This memory can be used to store data or execute programs. Although smaller in size than SDRAM, this on-chip memory provides faster access and lower latency. So, it is very suitable for handling small data.
- **Parallel I/O interface**
  There is a bus connection between the PIO peripheral and NIOS II through the Avalon bus. In this lab, we use switches as an input and LEDs as an output. Switch input is an 8-bit number that will be added to the accumulator. The LEDs will light up as output, displaying the current accumulator value. PIO is the interface between the NIOS II processor and the external hardware (i.e., FPGA logic). Using qsys, the base address of PIO is assigned.

- **Off-chip SDRAM**

  On-chip memory has a very limited storage capacity. So, 128 Mbytes of external memory provided by SDRAM is used in this lab. The SDRAM controller IP must be used to interface the SDRAM to the Avalon bus since the SDRAM itself cannot be directly connected to the bus, which is the Avalon memory-mapped bus.

  The IP SDRAM controller must be instantiated with a specific SDRAM parameter (Data Width, # of Rows, # of Columns, # of Chip Selects, # of Banks) in order to work to achieve the goals in this lab. In short, this SDRAM controller manages memory operations between the NIOS II processor and the SDRAM on the board.

- **PLL (Phase-Locked Loop) component**

  The clock signal for the SDRAM chip is provided by the PPL component. SDRAM requires precise timing synchronization. The PPL component is able to adjust for clock skew (i.e., timing difference due to delay in the circuit paths) to ensure the SDRAM receives a stable clock signal. This PPL will phase shift the external clock at the specific frequency at each clock. PPL is also used because there is a 3 ns delay required by SDRAM and the general clock.

- **System ID Checker**

  To ensure the compatibility between hardware and software, a system ID checker is utilized. This block works by assigning the unique ID to each hardware and preventing loading programs if the FPGA configuration does not match.

Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Tags	Opcode Name
		clk_in_reset	Reset Input	reset						
		clk	Clock Output	Double-click to export	clk_0					
		clk_reset	Reset Output	Double-click to export						
☑		nios2_gen2_0	Nios II Processor							
		clk	Clock Input	Double-click to export	clk_0					
		reset	Reset Input	Double-click to export	[clk]					
		data_master	Avalon Memory Mapped Master	Double-click to export	[clk]					
		instruction_master	Avalon Memory Mapped Master	Double-click to export	[clk]					
		irq	Interrupt Receiver	Double-click to export	[clk]	IRQ 0	IRQ 31			
		debug_reset_requ...	Reset Output	Double-click to export	[clk]					
		debug_mem_slave	Avalon Memory Mapped Slave	Double-click to export	[clk]	0x0000_1000	0x0000_17ff			
		custom_instructio...	Custom Instruction Master	Double-click to export						
☑		onchip_memory2_0	On-Chip Memory (RAM or ROM)...							
		clk1	Clock Input	Double-click to export	clk_0					
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk1]	0x0000_0000	0x0000_000f			
		reset1	Reset Input	Double-click to export	[clk1]					
☑		led	PIO (Parallel I/O) Intel FPGA IP							
		clk	Clock Input	Double-click to export	clk_0					
		reset	Reset Input	Double-click to export	[clk]					
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	0x0000_0070	0x0000_007f			
		external_connection	Conduit	led_wire						
☑		sdram	SDRAM Controller Intel FPGA IP							
		clk	Clock Input	Double-click to export	sdram_pl...					
		reset	Reset Input	Double-click to export	[clk]					
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	0x1000_0000	0x17ff_ffff			
		wire	Conduit	sdram_wire						
☑		sdram_pll	ALTPLL Intel FPGA IP							
		inclk_interface	Clock Input	Double-click to export	clk_0					
		inclk_interface_reset	Reset Input	Double-click to export	[inclk_inte...					
		pll_slave	Avalon Memory Mapped Slave	Double-click to export	[inclk_inte...	0x0000_0080	0x0000_008f			
		c0	Clock Output	Double-click to export	sdram_pll...					
		c1	Clock Output	sdram_clk	sdram_pll...					
☑		sysid_qsys_0	System ID Peripheral Intel FPGA...							
		clk	Clock Input	Double-click to export	clk_0					
		reset	Reset Input	Double-click to export	[clk]					
		control_slave	Avalon Memory Mapped Slave	Double-click to export	[clk]	0x0000_0098	0x0000_009f			
☑		switch	PIO (Parallel I/O) Intel FPGA IP							
		clk	Clock Input	Double-click to export	clk_0					
		reset	Reset Input	Double-click to export	[clk]					
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	0x0000_0060	0x0000_006f			
		external_connection	Conduit	switch_wire						
☑		key	PIO (Parallel I/O) Intel FPGA IP							
		clk	Clock Input	Double-click to export	clk_0					
		reset	Reset Input	Double-click to export	[clk]					
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	0x0000_0050	0x0000_005f			
		external_connection	Conduit	key_wire						

Figure 1 Hardware Component

2.1.2 Software Component

- **Blinker Code**

  For the blinking task, we wrote a simple C program to make the LEDG0 on the DE2-115 board blink by using the NIOS II processor. The first step is to initialize the pointer, LED_PIO, to have direct access to the LED PIO block. The address of PIO blocks corresponds to the address we assigned in the qsys file. Then, clear all the LEDs before entering the infinite loop. To clear all the LEDs, 0 is assigned to the LED's memory-mapped PIO address, and the LEDs are turned off. In the infinite loop, the LEDG0 will be turned on and off by toggling the LSB of the PIO block. By setting the LSB of the LED block to 1, the rightmost LED (i.e., LEDG0) will turn on, and vice versa. We set the software delay by implementation of a for loop to create a delay between LEDG0 turning on and off. So that observers (people/users) can observe the visible blinking of the LEDG0 DE2-115 board. Hence, the set LSB and clear LSB will be executed continuously in the infinite while loop.

- **Accumulator Program**

  We perform an 8-bit accumulator program on the DE2-115 board using the NIOS II processor. Overall, the C program contains 3 main steps, which are:

  - **Initialization**

    Initialize the memory-mapped pointers, LED_PIO, SWITCHES, and KEY, to access directly the hardware LEDs, switches, and KEY buttons. Then initialize several variables: reset, acc, and acc_int. These variables function to track the state of the button and accumulate value.

    Then, the infinite loop is entered:

  - **Input Handling**

    KEY[2] is a reset button. When the button is pressed, the accumulator, presented by the acc_int variable, will have a 0 value. This makes the LEDs turn off. Active-low logic is used.

    KEY[3] is an accumulate button. When the button is pressed, the switch input (presented by *SWITCH) will be added to the accumulated value (presented by acc_int). The overflow is handled by wrapping around at 256. Active-low logic is used.

    The “reset” and “acc” indicators serve to ensure that the press triggers only one action.

  - **Accumulator Logic & Display**

    The output of the accumulator will be displayed by the LEDs (i.e., LED_PIO block). The current accumulated value (presented by acc_int) is continuously written in the LED output to update the accumulated value. At the end, the buttons are released to allow the system to detect a new press.

2.2 Written Description of All .sv Modules

- **Module:** lab7
  Inputs: CLOCK_50, [3:0] KEY, [7:0] SW, [7:0]
  Outputs: LEDG, [12:0] DRAM_ADDR, [1:0] DRAM_BA, DRAM_CAS_N, DRAM_CKE, DRAM_CS_N, [3:0] DRAM_DQM, DRAM_RAS_N, DRAM_WE_N, DRAM_CLK
  inout: [31:0] DRAM_DQ
  Description: This is the top level. This module declares a soc module and integrates the lab7_soc to the FPGA, connecting the hardware components.
  Purpose: This module facilitates communication between the Nios II processor and peripherals. Apart from that, this module ensures that synchronization and routing are properly set between the processor and the external devices.

- **Module:** lab7_soc
  Inputs: input wire clk_clk, wire [1:0] key_wire_export, wire reset_reset_n, wire [7:0] switch_wire_export
  Outputs: output wire [7:0] led_wire_export, wire sdram_clk_clk, wire [12:0] sdram_wire_addr, wire [1:0] sdram_wire_ba, wire sdram_wire_cas_n, wire dram_wire_cke, wire sdram_wire_cs_n, wire [3:0] sdram_wire_dqm, wire sdram_wire_ras_n, wire sdram_wire_we_n,
  inout: wire [31:0] sdram_wire_dq,
  Description: This module is the System-on-Chip (SoC) that integrates the Nios II processor, memory interfaces (i.e., on-chip Ram and SDRAM), and peripheral I/O blocks (i.e., switches, keys, and LEDs). PPL and system ID are also included in this module. The entire connection is made via the Avalon memory-mapped interconnect bus.
  Purpose: The goal of this module is to make the Nios II processor able to perform accumulator tasks, namely reading switch inputs, adding them to the accumulator, and displaying the accumulated value on the LEDs. With this module, the Nios II processor and hardware can be well integrated.

2.3 Top Level Block Diagram

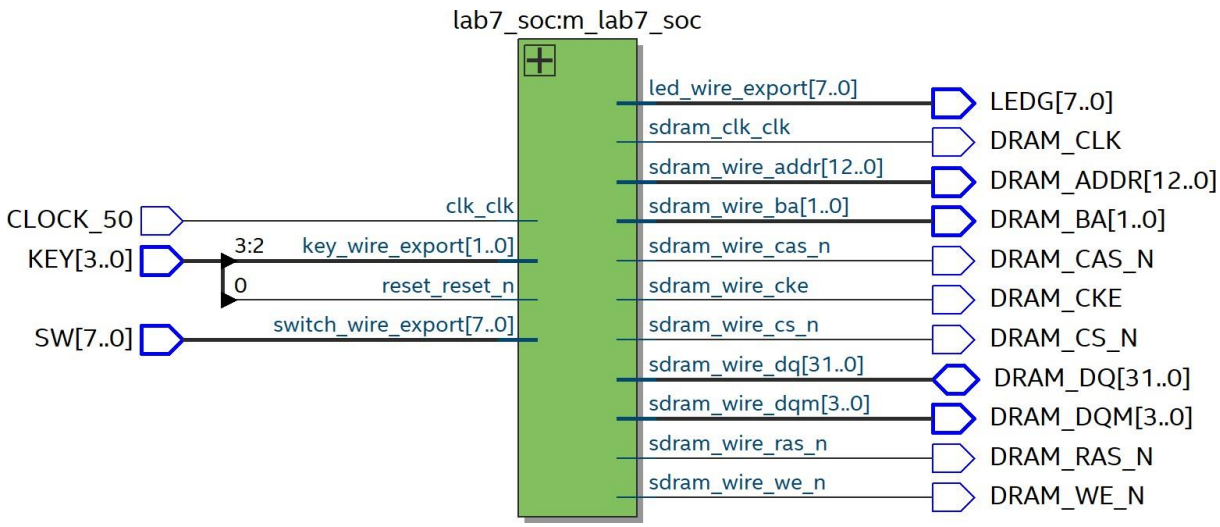

Figure 2 RTL Viewer: Top Level Block Diagram

3. Answers to all INQ Questions

1. **What are the differences between the Nios II/e and Nios II/f CPUs?**
   The Nios II/e is a slower (due to the higher clock cycle count per instruction) but smaller core (due to the minimal logic element and resources used), used where the microcontroller capability of the Nios is most important. Nios II/e is the economy version of the processor. On the other hand, Nios II/f is a faster but larger core used when performance is critical.

2. **What advantage might on-chip memory have for program execution?**
   On-chip memory is directly integrated into the FPGA. Therefore, the access time of on-chip memory is faster than off-chip memory (e.g., SDRAM). The speed of access to off-chip memory is slower due to longer data paths. Therefore, for important data with high access frequency, on-chip memory is suitable for use because its performance is fast and latency is reduced.

3. **Note the bus connections coming from the NIOS II; is it a Von Neumann, "pure Harvard", or "modified Harvard" machine and why?**
   Nios II is a Modified Harvard Machine because it uses separate buses for instruction and data transfer, although it has a shared memory for instructions and data. In short, there are two separate paths to transfer data. We are able to retrieve the instructions as data.
   Von Neumann: instructions and data are stored by the same memory and bus (shared).
   Pure Harvard: instructions and data have their own distinct buses that they use to store machine data in different memory units (distinct).
   Modified Harvard: instruction and data have their own distinct bus but its memory is shared.

4. **Note that while the on-chip memory needs access to both the data and program bus, the led peripheral only needs access to the data bus. Why might this be the case?**
   The LED peripheral only needs access to the data bus since it is an output peripheral for displaying the data. On the other hand, the on-chip memory needs access to both data and program bus because it stores both instructions and data.

5. **Why does SDRAM require constant refreshing?**
   SDRAM consists of capacitors and transistors, where the charge of the capacitors decay over time. Hence, by constant refreshing, SDRAM maintains the correct data.

6. **Make sure this is consistent with your above numbers; you will need to justify how you came up with 1 Gbit to your TA.**

SDRAM Parameter	Short Name	Parameter Value
Data Width	[width]	32 bits
# of Rows	[nrows]	13 bits
# of Columns	[ncols]	10 bits
# of Chip Selects	[ncs]	1 bit
# of Banks	[nbanks]	4 bits

Table 1 Parameter of SDRAM

   Justification: $32 \times 2^{13} \times 2^{10} \times 1 \times 4 = 128$ Mbytes

7. **What is the maximum theoretical transfer rate to the SDRAM according to the timings given?**
   The access time is 5.5 ns, and the data width is 32-bit. So, the maximum theoretical transfer rate to the SDRAM is (32 bits/5.5 ns) * (1 byte/8 bits) = 727 MB/s.

8. **The SDRAM also cannot be run too slowly (below 50 MHz). Why might this be the case?**
   Since the SDRAM comprises a capacitor, it needs to be frequently refreshed to maintain the data. If the SDRAM runs below 50 MHz, it will not refresh often enough, and the data loss might happen due to the decreasing voltage.

9. **Make another output by clicking clk c1 and verify it has the same settings, except that the phase shift should be -3ns. This puts the clock going out to the SDRAM chip (clk c1) 3ns ahead of the controller clock (clk c0). Why do we need to do this? Hint, check Altera Embedded Peripheral IP datasheet under SDRAM controller.**
   To accommodate for the propagation delay and metastability. The -3 ns phase shift ensures that the SDRAM's data is stable. Because to access data, the SDRAM controller needs time for synchronization. The phase shift gives the SDRAM controller time to stabilize the data access and ensures that the control signals are valid on the address pins.

10. **What address does the NIOS II start execution from? Why do we do this step after assigning the addresses?**
    NIOS II starts the execution at address x0200 0000 of SDRAM. This is because our intention is to go to the first instruction on that address and avoid the irrelevant memory regions.

11. **You must be able to explain what each line of this (very short) program does to your TA. Specifically, you must be able to explain what the volatile keyword does (line 8), and how the set and clear functions work by working out an example on paper (lines 13 and 16). This question is referring to the blinker code.**

```
1  // Main.c - makes LEDG0 on DE2-115 board blink if NIOS II is set up correctly
4
5  int main()
6  {
7      int i = 0;
8      volatile unsigned int *LED_PIO = (unsigned int*)0x20; //make a pointer to acc
9
10     *LED_PIO = 0; //clear all LEDs
11     while ( (1+1) != 3) //infinite loop
12     {
13         for (i = 0; i < 100000; i++); //software delay
14         *LED_PIO |= 0x1; //set LSB
15         for (i = 0; i < 100000; i++); //software delay
16         *LED_PIO &= ~0x1; //clear LSB
17     }
18     return 1; //never gets here
19 }
20
```

Figure 3 Blinker Code

Line 7: Create the counter variable I and initialize it to zero.

Line 8: Create the pointer LED_PIO that points to the memory address of LEDs. The volatile keyword is useful to prevent optimization by reading data from memory when the variable is used. By using the volatile keyword, the compiler knows that the object may be changed at any time, and the compiler cannot make assumptions about the object.

Line 10: Clear or turn off all the LEDs by assigning a zero to the LED register.

Line 11: Starting the infinite loop.

The set and clear function works by setting the LSB of LED_PIO to one and zero, alternately with the software delay between them to create a visible blinking. The explanation of each line is:

Line 13: The long loop for software delay.

Line 14: Set the LSB of the LED to 1, and the rightmost LED will turn on. The code: *LED_PIO |= 0x1 uses the OR logic to set the LSB of LED to 1. (SET FUNCTION).

Line 15: The long loop for software delay.

Line 16: Set the LSB of the LED to 0, and the rightmost LED will turn off. The code: *LED_PIO &= ~0x1 uses the OR logic to set the LSB of LED to 0. (CLEAR FUNCTION).

Line 18: The program will never reach this line due to the existence of the infinite loop. However, if it is reached, the program will return 1.

12. **Look at the various segment (.bss, .heap, .rodata, .rwdata, .stack, .text), what does each section mean? Give an example of C code which places data into each segment, e.g. the code: const int my_constant[4] = {1, 2, 3, 4} will place 1, 2, 3, 4 into the .rodata segment.**

Segment	Meaning	Example (C code)
.bss	Contains statically allocated variables and constants that have been declared but not yet assigned a value (i.e., unitialized).	int x;
.heap	Region for dynamically allocated memory.	int pointer= (int)malloc(sizeof(int));
.rodata	Read only data, region for constants	const int x = 0;
.rwdata	Read-Write data that can be modified during execution	int x = 1;
.stack	For allocated variables and function calls.	int func(int a, intb)
.text	Region of text/strings	char x = “text”;

Table 2 Various Segment

4. Postlab Question

4.1 Document the Design Resources and Statistics

LUT	2270
DSP	0
Memory (BRAM)	36864
Flip-Flop	1977
Frequency	80.63 MHz
Static Power	102.06 mW
Dynamic Power	40.64 mW
Total Power	204.07 mW

Table 3 Design Resources and Statistics

5. Conclusion

5.1. Functionality of The Design

Designing the 8-bit input accumulator by using the NIOS II processor to manage the SoC tasks and peripheral interfacing. The input switches are added to the accumulator, and the LEDs successfully display the accumulated value. The overflow and reset are also handled well according to the lab manual requirements. In doing this lab, we did not face any significant difficulties. However, in the beginning, it took us some time to learn how SDRAM works. By reading the lab manual and searching the internet, we finally figured out how SDRAM works and successfully completed this lab.

5.2. Suggestions for Improvement and Feedback

The Lab Manual and other materials are very clear, straightforward, and informative. The guides and instructions are easy to understand and leave no ambiguity in their interpretation. Overall, this lab is very useful and has improved our knowledge of the NIOS II Processor, specifically the fundamentals of memory-mapped I/Os and implemented a simple SoC interfacing with peripherals. The tutorials and guidelines provided were thoroughly detailed and helpful in completing this lab.